

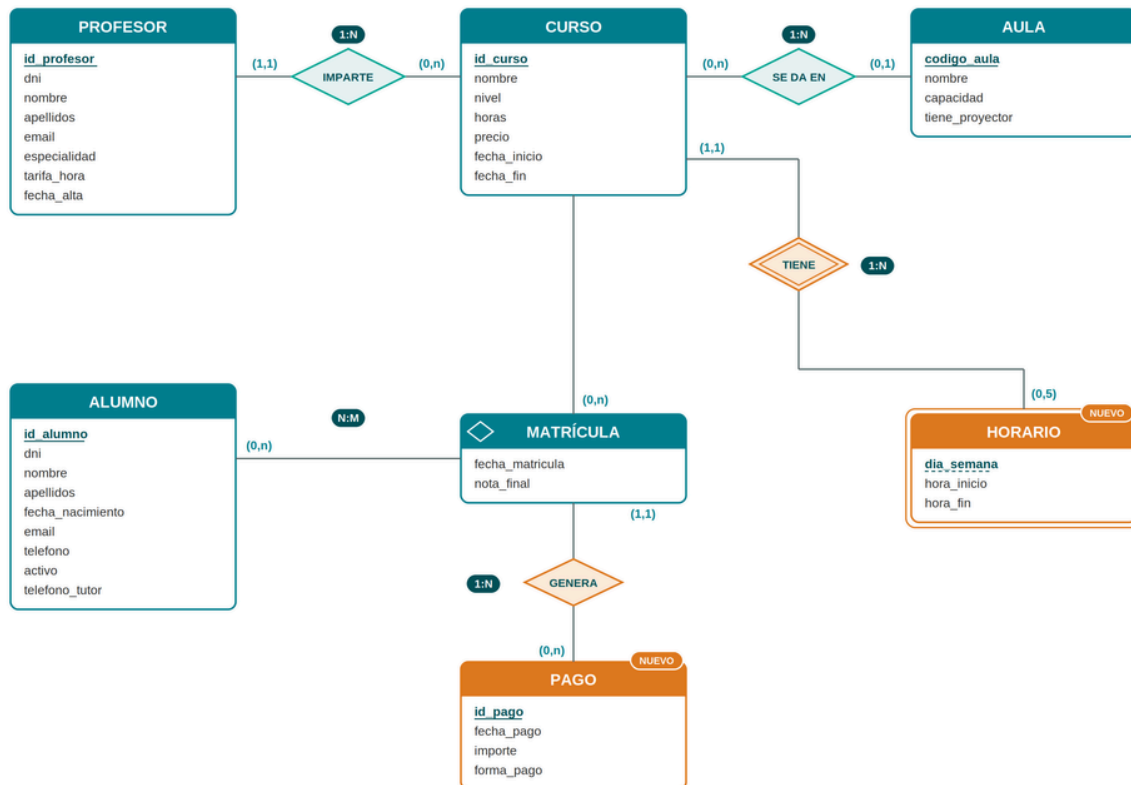
Horarios y pagos

Academia Código Norte · el modelo completo

Duración	Qué vas a aprender	Necesitas
1–2 sesiones (~1,5 h)	Entidad débil, ON DELETE CASCADE, clave ajena compuesta e índices	Haber terminado la práctica 4

Dónde estamos

Completamos el modelo. El **HORARIO** de un curso no tiene sentido sin su curso: es una entidad **débil**, y si se borra el curso, su horario se va con él. Los **PAGOS** no son de un alumno ni de un curso sueltos: son de una **matrícula** concreta.



En naranja, lo nuevo de esta práctica.

Lo nuevo de hoy

- **ON DELETE CASCADE:** si se borra el curso, se borran solas todas sus filas de horario.
- **Clave ajena compuesta:** pago apunta a matricula, y como la clave de matrícula tiene dos columnas, la clave ajena también tiene dos.
- **Índice:** como el índice de un libro. Hace que buscar por una columna sea mucho más rápido.
- **TIME:** un tipo de dato para horas, escrito como '17:00'.

La tabla HORARIO

```

CREATE TABLE horario (
    id_curso    INT      NOT NULL,
    dia_semana  INT      NOT NULL CHECK (dia_semana BETWEEN 1 AND 5),
    hora_inicio TIME     NOT NULL,
    hora_fin    TIME     NOT NULL,
    CONSTRAINT pk_horario PRIMARY KEY (id_curso, dia_semana),

```

```

CONSTRAINT ck_horario_horas CHECK (hora_fin > hora_inicio),

CONSTRAINT fk_horario_curso FOREIGN KEY (id_curso)

REFERENCES curso (id_curso) ON DELETE CASCADE

);

```

Qué guarda cada columna:

Columna	Tipo	Reglas	Qué guarda
id_curso	INT	NOT NULL	De qué curso es (clave ajena)
dia_semana	INT	NOT NULL CHECK (dia_semana BETWEEN 1 AND 5)	1 = lunes ... 5 = viernes
hora_inicio	TIME	NOT NULL	Hora de empezar
hora_fin	TIME	NOT NULL	Hora de acabar

Y sus reglas de tabla:

Nombre	Regla	Qué hace
pk_horario	PRIMARY KEY (id_curso, dia_semana)	Un curso tiene como mucho una sesión cada día
ck_horario_horas	CHECK (hora_fin > hora_inicio)	No se acaba antes de empezar
fk_horario_curso	FOREIGN KEY (id_curso) REFERENCES curso (id_curso) ON DELETE CASCADE	Si se borra el curso, se borra su horario

La tabla PAGO

```
CREATE TABLE pago (  
    id_pago      INT          PRIMARY KEY,  
    id_alumno    INT          NOT NULL,  
    id_curso     INT          NOT NULL,  
    fecha_pago   DATE         NOT NULL DEFAULT CURRENT_DATE,  
    importe      NUMERIC(7,2) NOT NULL CHECK (importe > 0),  
    forma_pago   VARCHAR(15)  NOT NULL CHECK (forma_pago IN ('Efectivo', 'Tarjeta', 'Transferencia',  
    'Bizum')),  
    CONSTRAINT fk_pago_matricula FOREIGN KEY (id_alumno, id_curso)  
        REFERENCES matricula (id_alumno, id_curso) ON DELETE RESTRICT  
);
```

Qué guarda cada columna:

Columna	Tipo	Reglas	Qué guarda
id_pago	INT	PRIMARY KEY	Número del pago
id_alumno	INT	NOT NULL	Parte 1 de la matrícula
id_curso	INT	NOT NULL	Parte 2 de la matrícula
fecha_pago	DATE	NOT NULL DEFAULT CURRENT_DATE	Cuándo se pagó
importe	NUMERIC(7,2)	NOT NULL CHECK (importe > 0)	Cantidad en euros
forma_pago	VARCHAR(15)	NOT NULL CHECK (forma_pago IN ('Efectivo', 'Tarjeta', 'Transferencia', 'Bizum'))	Cómo se pagó

Lo nuevo es la última regla, `fk_pago_matricula`: una clave ajena de **dos columnas** que apunta a las dos columnas de la clave de `matricula`. Así solo se puede pagar una matrícula que existe.

Los índices

Al final, se crean los dos índices:

```
CREATE INDEX idx_alumno_apellidos ON alumno (apellidos);
```

```
CREATE INDEX idx_pago_matricula ON pago (id_alumno, id_curso);
```

Nombre	Tabla (columnas)	Para qué
idx_alumno_apellidos	alumno (apellidos)	Recepción busca alumnos por apellido a todas horas
idx_pago_matricula	pago (id_alumno, id_curso)	Buscar rápido los pagos de una matrícula

Paso a paso

Hoy tendrás **7 consultas guardadas** en el SQL Editor, cada una con su nombre (igual que en la práctica 1):

Nombre de la consulta	Qué tiene dentro
P5 · 1 · Tabla horario	EI CREATE TABLE horario
P5 · 2 · Tabla pago	EI CREATE TABLE pago
P5 · 3 · Índice apellidos	CREATE INDEX idx_alumno_apellidos ...
P5 · 4 · Índice pagos	CREATE INDEX idx_pago_matricula ...

P5 · 5 · Datos de prueba	Los INSERT de horarios y pagos
P5 · 6 · Intenta romperla	Las tres pruebas que deben fallar
P5 · 7 · Prueba CASCADE	El bloque BEGIN ... ROLLBACK

1. Crea cada consulta **en el orden de la tabla**: **New query**, pega, **Run** y, si sale «Success», renómbrala (tres puntos → **Rename query**).
2. En el **Table Editor** deben aparecer ya las **7 tablas** de la academia.
3. Ejecuta las pruebas de **Intenta romperla** una a una (selecciona cada una y pulsa **Run**) y la de **CASCADE**.

Datos de prueba

```
INSERT INTO horario (id_curso, dia_semana, hora_inicio, hora_fin) VALUES
```

```
(1, 1, '17:00', '19:00'),
```

```
(1, 3, '17:00', '19:00'),
```

```
(2, 2, '18:00', '20:30'),
```

```
(3, 2, '16:00', '18:00'),
```

```
(3, 4, '16:00', '18:00'),
```

```
(4, 5, '17:00', '20:00');
```

```
INSERT INTO pago (id_pago, id_alumno, id_curso, fecha_pago, importe, forma_pago) VALUES
```

```
(1, 1, 1, '2026-09-20', 90.00, 'Bizum'),
```

```
(2, 1, 3, '2026-09-20', 120.00, 'Tarjeta'),
```

```
(3, 2, 2, '2026-09-22', 150.00, 'Transferencia'),
```

```
(4, 3, 1, '2026-09-25', 90.00, 'Efectivo');
```

Comprueba

horario tiene 6 filas y pago tiene 4.

Para ver los índices: `SELECT indexname FROM pg_indexes WHERE schemaname = 'public';`
Salen los tuyos y muchos más: PostgreSQL crea uno solo por cada clave primaria y cada UNIQUE.

Intenta romperla

#	Sentencia	Debe fallar porque...
1	<pre>INSERT INTO pago (id_pago, id_alumno, id_curso, importe, forma_pago) VALUES (5, 4, 1, 50, 'Bizum');</pre>	Leo no está matriculado en Python (fk_pago_matricula)
2	<pre>INSERT INTO pago (id_pago, id_alumno, id_curso, importe, forma_pago) VALUES (5, 2, 2, 10, 'Cheque');</pre>	'Cheque' no es una forma de pago válida (CHECK)
3	<pre>INSERT INTO horario VALUES (2, 6, '10:00', '12:00');</pre>	El día 6 sería sábado (CHECK)

Comprueba el CASCADE

El curso 4 (Next.js) tiene horario pero ningún alumno. Ejecuta el bloque **entero**:

```
BEGIN;
```

```
DELETE FROM curso WHERE id_curso = 4;
```

```
SELECT count(*) AS horarios_del_curso_4 FROM horario WHERE id_curso = 4;
```

```
ROLLBACK;
```

Comprueba

Sale 0: al borrar el curso, su horario se ha borrado solo. El **ROLLBACK** lo deshace todo al final.

Piensa

¿Qué problema habría si **pago** tuviera dos claves ajenas separadas, una a **alumno** y otra a **curso**? Piensa en la prueba 1.

Si los índices hacen las búsquedas más rápidas, ¿por qué no ponemos uno en cada columna?

¿Te has equivocado? Empieza de nuevo

Ejecuta esto y vuelve a lanzar tu código de esta práctica desde el principio. Solo borra lo de **esta** práctica: lo de las anteriores no se toca.

```
DROP TABLE IF EXISTS horario, pago CASCADE;
```

```
DROP INDEX IF EXISTS idx_alumno_apellidos;
```

Qué entregas

Captura de la lista de consultas con las de la práctica 5 ([P5](#) · ...) y captura del Table Editor con las 7 tablas.

Anclaje curricular · RD 405/2023 · Módulo 0484 · RA2 · **criterios b, c, d, e y h**